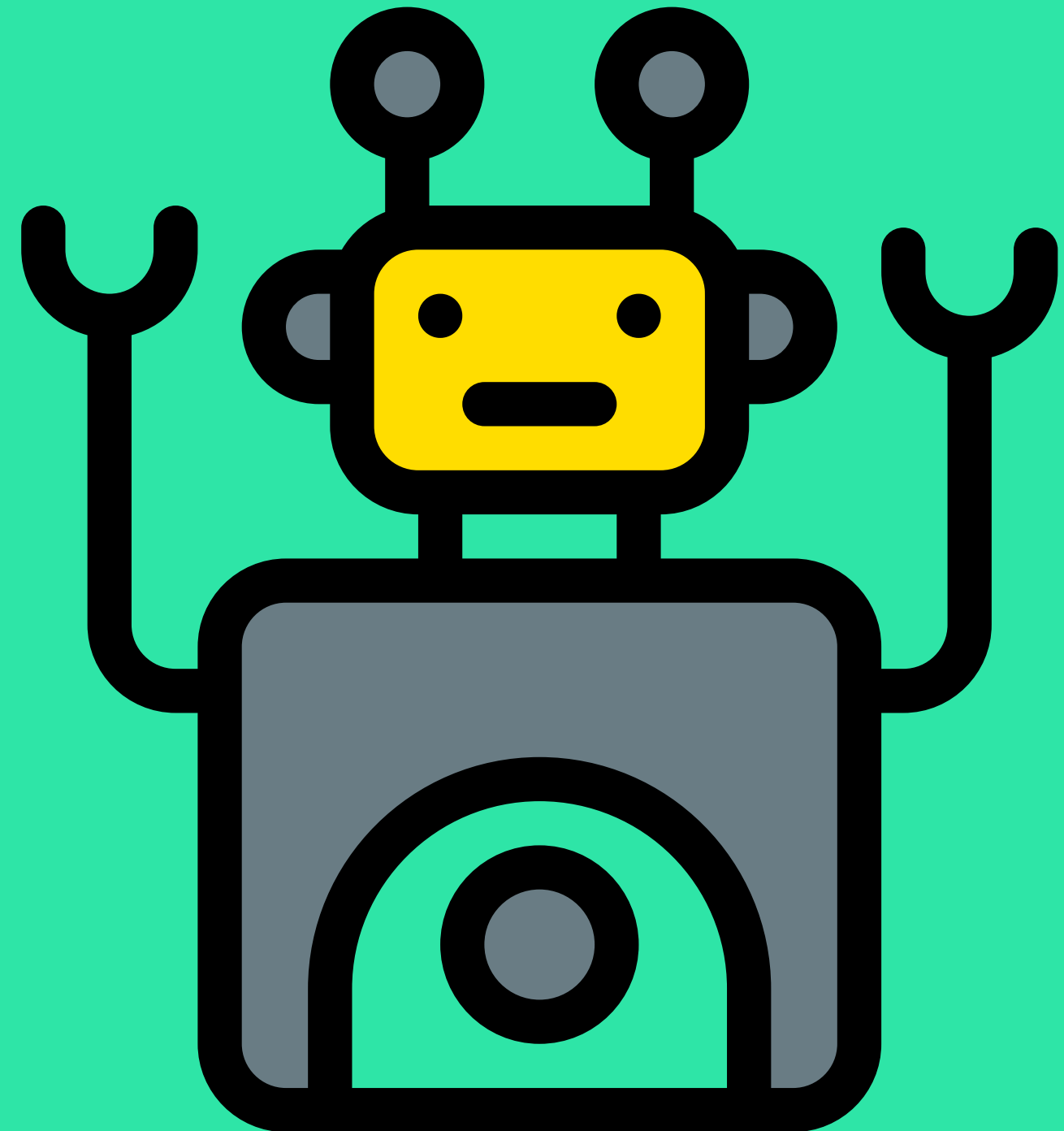


TIM ASISTEN
PRAKTIKUM

SHA-3

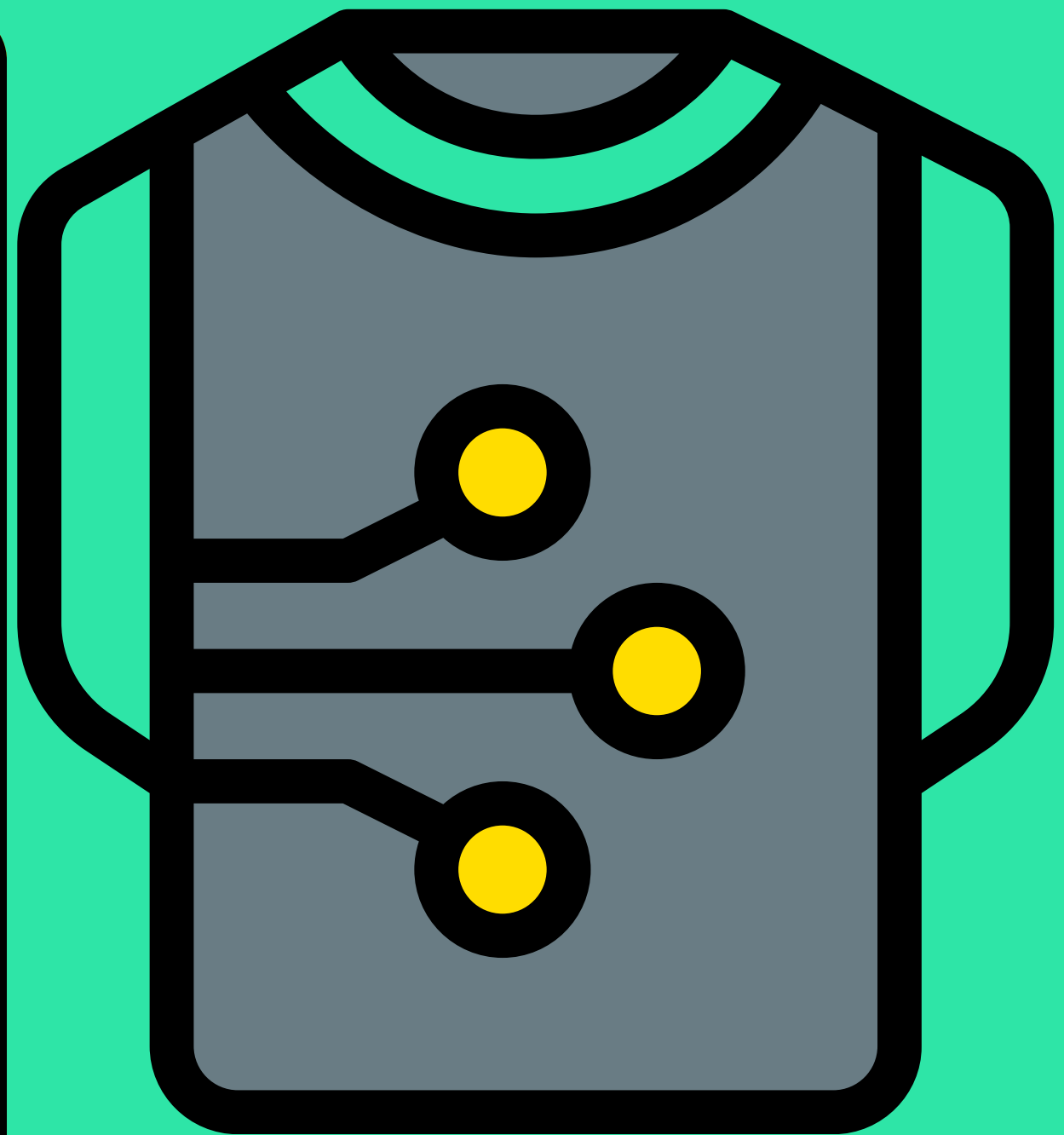
SECURE HASH ALGORITHM 3



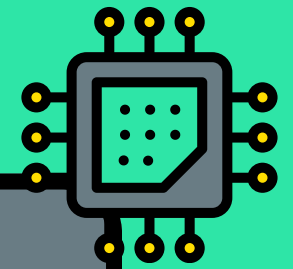
INTRODUCTION

SHA-3 (Secure Hash Algorithm 3) adalah fungsi hash kriptografi yang mengambil input (pesan) dengan panjang berapapun dan menghasilkan output (hash/digest) dengan panjang tetap sesuai variannya.

Algoritma ini beroperasi menggunakan operasi matematika tingkat bit (bitwise) murni.



STEPS



1. PENAMBAHAN PADDING BITS

SHA-3 memproses pesan dalam **blok State (b) = 1600-bit**. Tetapi, state (b) dibagi menjadi dua: **r (rate) & c (capacity)**. Panjang pesan = panjang r.

2. INISIALISASI STATE

Pada awalnya, kita tentukan terlebih dahulu, **Mau seberapa Panjang Hash (d) nya?**
Panjang c (capacity) = 2 * d
 $r = b - c$

Isi dari **r & c = 0** untuk inisialisasi awal

3. ABSORBING

Fase Absorbing dapat diartikan seperti **Spons yang menyerap Air**. Tapi di sini, **Spons adalah State (b)** dan **Air yang diserap adalah Pesan Asli**. Pesan asli diserap ke bagian r (Rate).

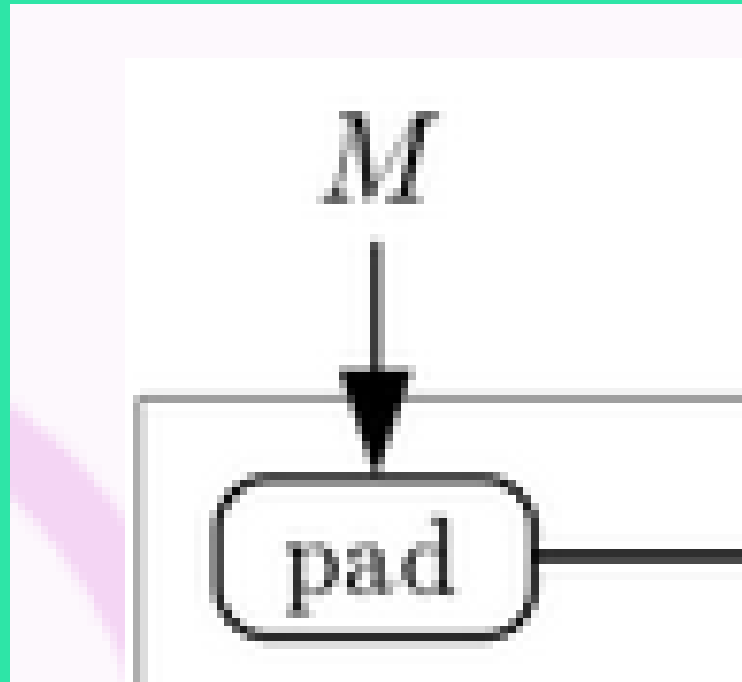
Lalu, State diolah pada Fungsi Keccak-f

4. SQUEEZING

Setelah pesan asli semuanya sudah “**diserap**” oleh State, maka **Hash Value sudah bisa didapatkan!**
Kita tinggal melakukan **pemotongan dari kiri sepanjang bit yang dibutuhkan** (misal 256-bit, maka potong sepanjang 256-bit bagian r (rate) untuk mendapatkan Hash Value nya).

Bagaimana kalau r (rate) < d (panjang hash value)? Lakukan **Fungsi Keccak-f lagi!** Supaya dapat Hash Value yang berbeda dan di **satukan dengan yang sebelumnya**

1. PENAMBAHAN PADDING BITS



Input Awal (ASCII)

'a' = 0x61 = 01100001

'b' = 0x62 = 01100010

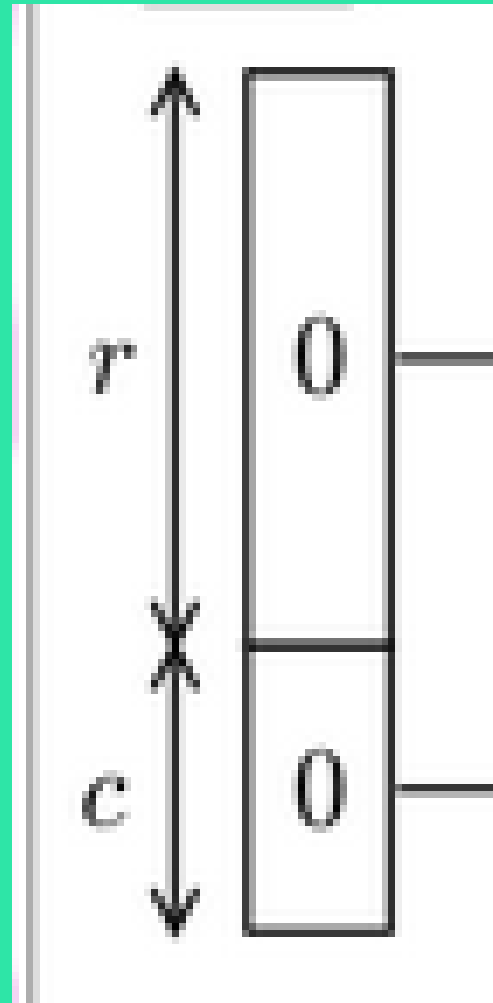
'c' = 0x63 = 01100011



Padding

1. **Tambahkan bit '1'** di akhir pesan: 01100001 01100010 01100011 **1**
2. **Tambahkan bit '0'** sampai sepanjang r (rate) . Jadi, 01100011 10000000 00.....
3. **Lalu**, diakhir **tutup dengan '1'**. 10000.....01
4. **Jadi totalnya** adalah sepanjang bit pada rate

2. INISIALISASI STATE



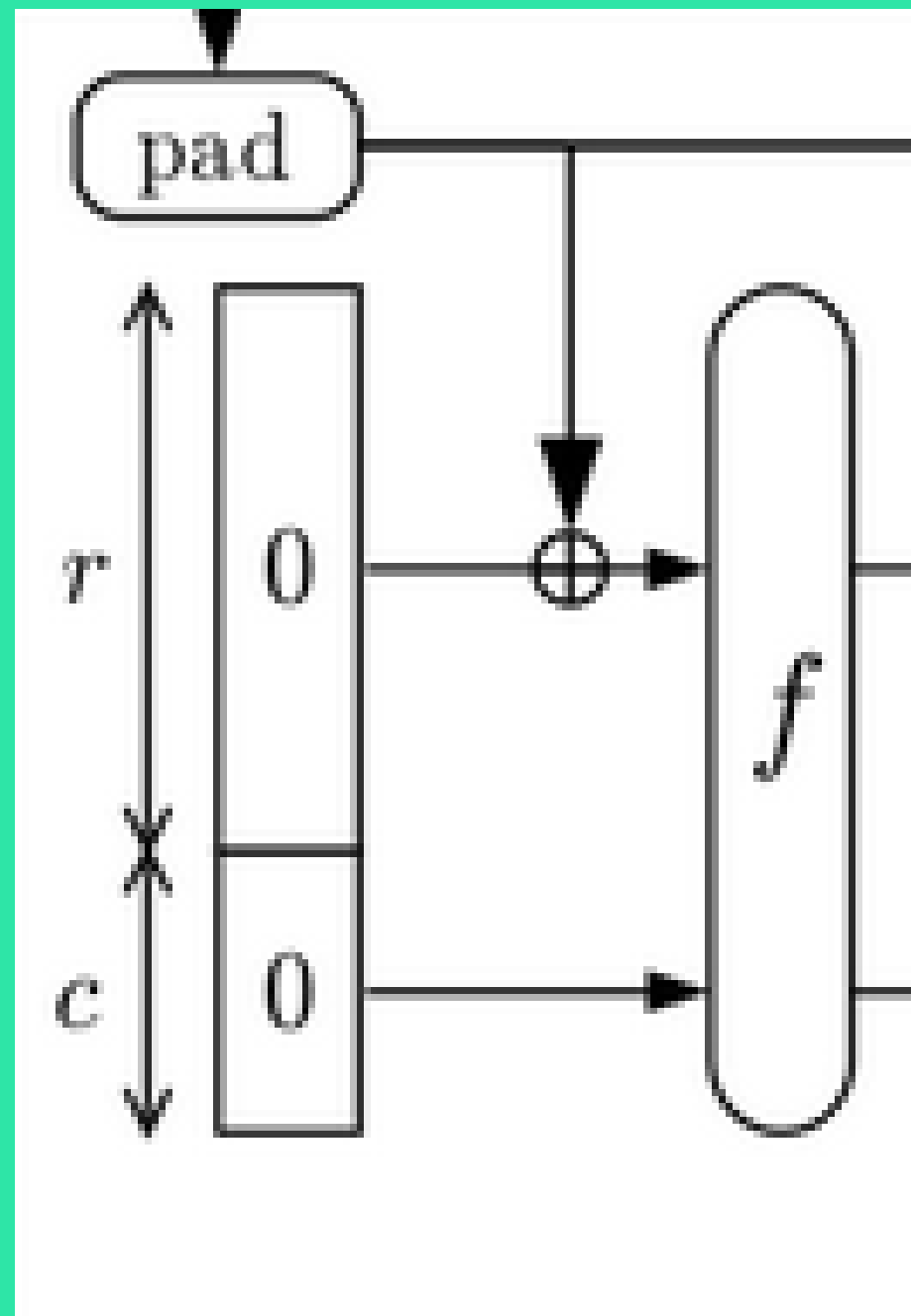
```
# Initialization  
S[x,y] = 0,
```

Inisialisasi State (b)

- State (b) terdiri dari **1600-bit**.
($b = r + c$)
- r (rate) terdiri dari $r = b - c$
- c (capacity) terdiri dari $c = 2 * d$
(panjang hash)

Pada tahap inisialisasi, semuanya dibuat = 0

3. ABSORBING

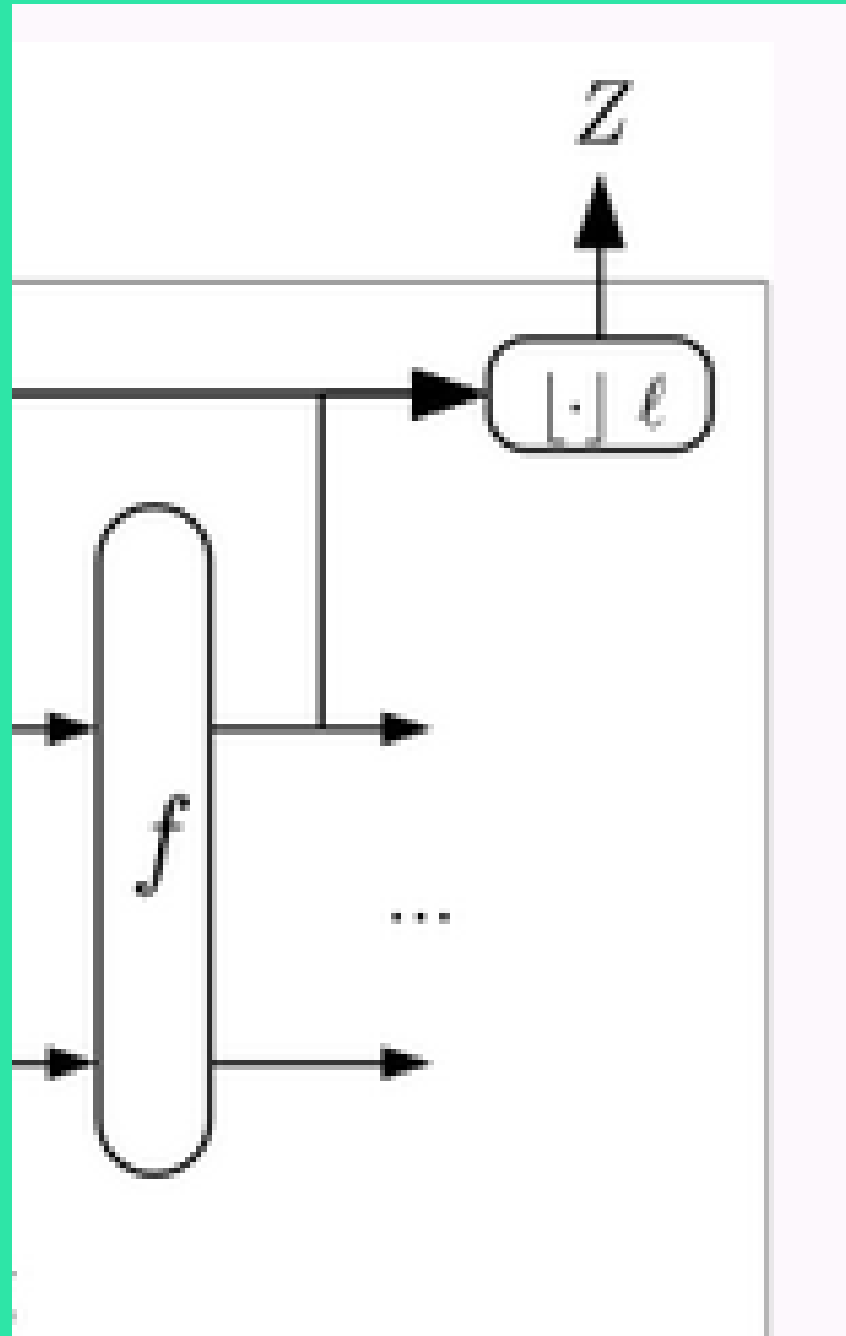


Absorbing

Pada tahap Absorbing, sebelum State kita “aduk aduk” pada Fungsi Hash (Keccak-f), kita lakukan “Penyerapan Pesan” terlebih dahulu.

1. Pesan yang sudah di padding, memiliki panjang = r . Maka dari itu, bisa dilakukan operasi bitwise XOR untuk mencampurkan r dengan pesan asli.
2. Setelah pesan “diserap”, State diproses pada fungsi Keccak-f.

4.SQUEEZING

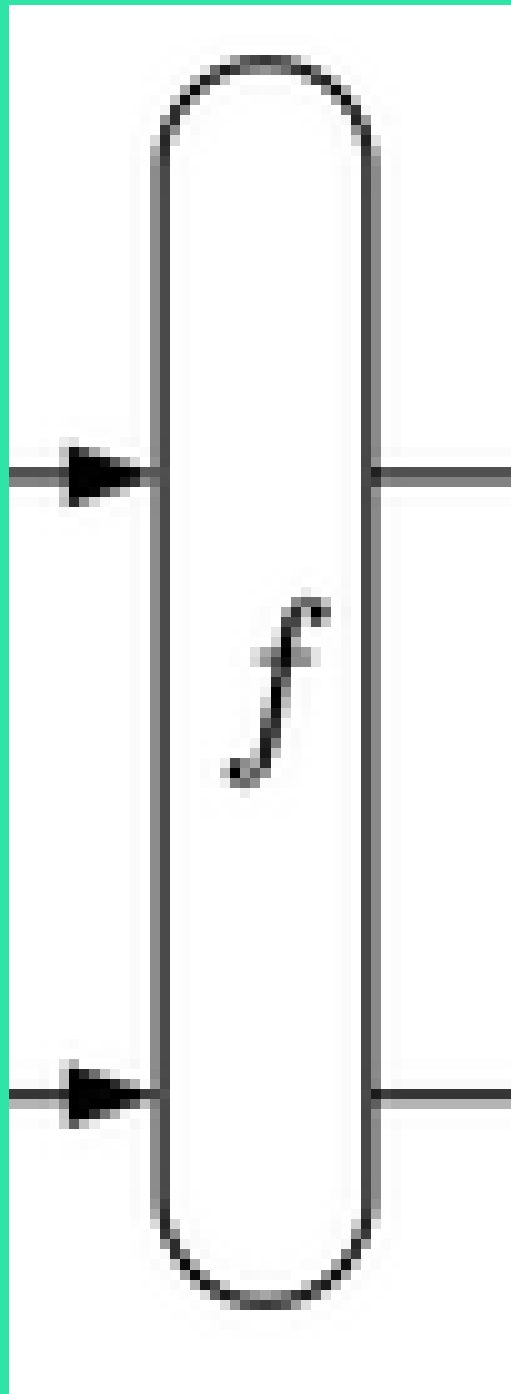


Squeezing

Tahap Squeezing dilakukan ketika **semua pesan sudah diserap** pada proses Absorbing.

1. Output fungsi (b , State yang sudah diaduk-aduk) dapat kita potong bagian r untuk dijadikan Hash Value
2. Misal $r = 1088$ bit, dan panjang hash (d) = 256. Maka tinggal ambil 256 bit dari r , urutan dari kiri.
3. Tapi, jika $r < d$ (misal, $r = 210$ bit, tapi $d = 512$), maka kekurangan $512 - 210 = 302$ bit, maka kita masukkan kembali State (b) ke Fungsi Keccak- f untuk mendapatkan r yang berbeda.
4. Lakukan concat (satuin), satukan 210 bit (dari r sebelumnya) dengan 302 bit (r sekarang).

EXTRA. FUNGSI KECCAK-F



Keccak-F

Bagaimana dengan Fungsi Keccak-f, memang bagaimana cara SHA-3 “mengaduk” State (b)?

1. Sebelumnya, $b = 1600$ -bit sejajar, anggap array 1 dimensi sebanyak 1600.
2. Di fungsi Keccak-f, di-ubah representasi datanya menjadi **array 3D. Dimensi: $5 \times 5 \times 64$ (1600)**
3. **Lalu** array tersebut akan dilalui 5 tahapan, ($\theta, \rho, \pi, \chi, \iota$) [Theta, Rho, Pi, Chi, Iota], sebanyak 24 putaran/ronde.

TIM ASISTEN
PRAKTIKUM

THANK YOU

